

Universal Image Restoration, Corruption-Aware Reconstruction, and Conditional Image Translation: A Comprehensive Study

Ayesha Noor

Department of Computer Science

Registration No: 23i-0736

Abstract—This report presents a comprehensive investigation into image restoration and translation using deep learning architectures across four interconnected tasks. Task 1 develops a Universal Denoising Autoencoder (UDAE) with a 16×16 spatial bottleneck, achieving an overall PSNR of 25.52 dB and SSIM of 0.8128. Task 2 introduces a corruption-aware pipeline consisting of a convolutional classifier, a hard routing mechanism, and three U-Net specialists, achieving routed PSNR of 36.05 dB. Task 3 extends Task 2 into a jointly trained soft mixture-of-experts that replaces hard routing with temperature-scaled softmax weighting, improving to PSNR of 36.47 dB and SSIM of 0.918. Task 4 addresses photo-to-sketch translation using a conditional GAN with a PatchGAN discriminator on the FS2K dataset, achieving SSIM of 0.481. All models are exported to ONNX format for deployment. Results demonstrate that preserving spatial structure in latent representations and jointly optimizing routing and reconstruction networks substantially improve restoration quality.

Index Terms—Image restoration, denoising autoencoder, U-Net, mixture-of-experts, conditional GAN, ONNX deployment.

I. INTRODUCTION

Image corruption is a pervasive problem in real-world computer vision applications, arising from sensor noise, motion blur, occlusion, and transmission errors. Restoring corrupted images to their clean form is essential for downstream tasks such as recognition, detection, and analysis. This report presents four interconnected tasks that progressively develop a comprehensive image restoration and translation system.

Task 1 establishes a universal image restoration baseline using a denoising autoencoder with a meaningful information bottleneck. Task 2 introduces corruption-aware routing with specialist networks. Task 3 extends this to a jointly trained soft mixture-of-experts. Task 4 addresses photo-to-sketch translation using conditional GANs.

II. TASK 1: UNIVERSAL IMAGE RESTORATION

A. Problem Formulation

The objective of Task 1 was to develop a universal image restoration model capable of reconstructing a clean image from multiple types and severities of image corruption. The model operates through a meaningful information bottleneck so that restoration cannot rely on unconstrained identity mapping.

The restoration problem is formulated as an autoencoder. A corrupted image is provided to the encoder, compressed

into a low-dimensional latent representation, and reconstructed by a decoder. The clean image serves as the reconstruction target. Three corruption families are considered: salt-and-pepper noise, Gaussian blur, and rectangular occlusion.

B. Dataset Preparation

Image data were resized to 128×128 pixels and stored as RGB images. Training corruption was generated dynamically, allowing each training sample to receive a newly sampled corruption configuration during loading. The training dataset supported clean images, salt-and-pepper noise, Gaussian blur, and occlusion. Horizontal flipping was applied as augmentation before corruption generation.

For validation and testing, deterministic manifests were generated so that corruption type, severity, parameters, and random seed were fixed. The test manifest contained ten entries per test image: one clean reference and three severity levels for each of the three corruption types.

TABLE I
TASK 1 DATASET AND CORRUPTION CONFIGURATION

Property	Value
Image resolution	128×128
Image channels	3 (RGB)
Training corruption	Runtime sampled
Validation	Deterministic manifest
Test images	3,669
Test entries	36,690
Corruption types	Salt-and-pepper, blur, occlusion
Severities per corruption	Low, medium, high

C. Architecture Design

The restoration network was implemented as a Universal Denoising Autoencoder (UDAE). Two bottleneck layouts were investigated: a vector bottleneck and a spatial bottleneck. The final architecture used a 16×16 spatial latent grid containing 4,096 values. The encoder reduced input resolution from 128×128 to 16×16 , while the decoder progressively upsampled back to 128×128 .

The encoder used convolution, batch normalization, and ReLU activation. The decoder used nearest-neighbour upsampling followed by convolutional blocks to avoid checkerboard artefacts. No skip connections were used between encoder

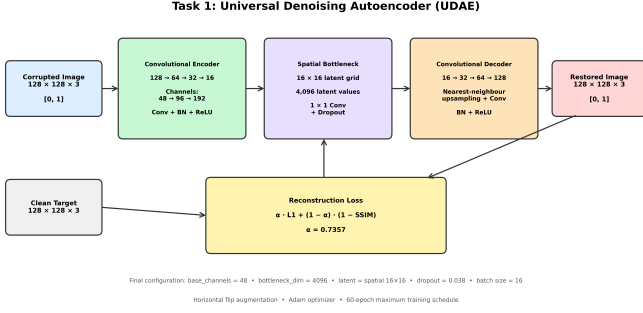


Fig. 1. Final Task 1 UDAE architecture using a 16×16 spatial latent grid containing 4,096 latent values.

and decoder, forcing all reconstructed information through the 4,096-value bottleneck.

D. Architecture Diagnostic

Four controlled designs were evaluated to determine whether the spatial structure of the latent representation was useful.

TABLE II
CONTROLLED ARCHITECTURE DIAGNOSTIC FOR TASK 1

Run	Latent Design	Compression	Params	PSNR/SSIM
A	Vector, 1024	$48 \times$	21.9M	17.77/0.440
B	Spatial 8×8 , 1024	$48 \times$	4.0M	20.69/0.656
C	Spatial 8×8 , 4096	$12 \times$	4.0M	22.76/0.683
D	Spatial 16×16 , 4096	$12 \times$	1.0M	23.69/0.749

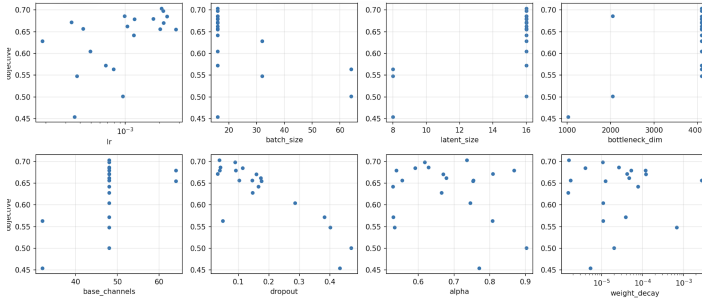


Fig. 2. Qualitative comparison of the investigated vector and spatial bottleneck designs.

The comparison showed that the spatial arrangement of the latent representation had a substantial effect on reconstruction quality. Run B improved PSNR by approximately 2.9 dB compared with the vector design while using substantially fewer parameters. Run D achieved a validation PSNR of 23.69 dB and SSIM of 0.749 while using approximately 1.0M parameters. This experiment provided the basis for removing the vector architecture from the subsequent Optuna search.

E. Loss Function

The autoencoder was trained using a combination of pixel-level L_1 reconstruction error and structural similarity:

$$\mathcal{L} = \alpha \mathcal{L}_{L_1} + (1 - \alpha)(1 - \text{SSIM}), \quad (1)$$

where $\alpha = 0.7357065$ controls the relative contribution of the pixel-level reconstruction term and the structural similarity term.

F. Training Procedure

The final model was trained for a maximum of 60 epochs using mixed precision and batch size 16. Key hyperparameters: learning rate 0.0020897, base channels 48, spatial latent size 16×16 , dropout 0.03787, weight decay 1.58×10^{-6} , early-stopping patience 12. The best checkpoint was obtained at epoch 51.

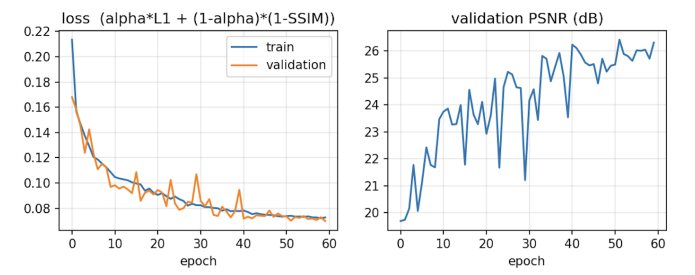


Fig. 3. Training and validation curves for the final Task 1 UDAE training run.

G. Optuna Hyperparameter Search

Optuna was used to search for the final configuration after the architecture diagnostic established the spatial bottleneck as preferred. Twenty-five trials were executed (20 completed, 5 pruned). The search considered latent grid size $\{8, 16\}$, bottleneck dimension $\{1024, 2048, 4096\}$, base channels, learning rate, batch size, dropout, $\alpha \in [0.5, 0.95]$, and weight decay $\in [10^{-6}, 10^{-2}]$.

TABLE III
BEST TASK 1 OPTUNA CONFIGURATION

Hyperparameter	Selected Value
Objective	0.7032
Learning rate	0.0020897
Batch size	16
Base channels	48
Dropout	0.03787
α	0.73571
Latent size	16×16
Bottleneck dimension	4096
Weight decay	1.58×10^{-6}

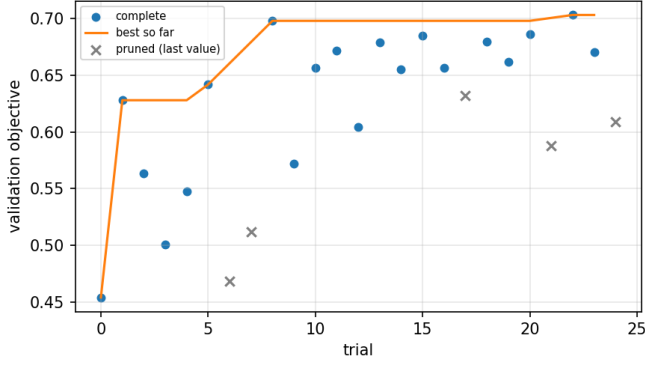


Fig. 4. Optuna optimization history for the Task 1 hyperparameter search.

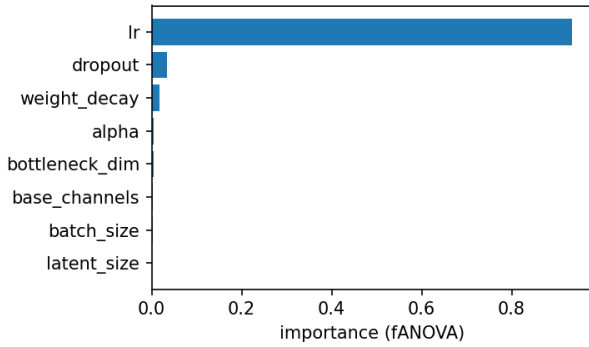


Fig. 5. Optuna hyperparameter importance for the Task 1 search.

TABLE IV
FINAL TASK 1 TEST-SET RESULTS BY CORRUPTION SEVERITY

Corruption	Sev.	PSNR _{in}	SSIM _{in}	PSNR _{out}	SSIM _{out}	Δ PSNR
Clean	None	—	—	27.807	0.871	—
Salt-pepper	Low	20.135	0.602	27.725	0.868	+7.590
	Med.	15.877	0.340	27.552	0.860	+11.675
	High	13.139	0.204	27.170	0.845	+14.031
Gaussian blur	Low	32.437	0.943	27.808	0.864	-4.629
	Med.	26.837	0.805	27.290	0.846	+0.453
	High	24.399	0.691	24.799	0.729	+0.399
Occlusion	Low	16.688	0.862	24.043	0.815	+7.355
	Med.	13.285	0.726	21.775	0.758	+8.490
	High	10.712	0.537	19.262	0.670	+8.550

TABLE V
OVERALL TASK 1 TEST-SET PERFORMANCE

Corruption	PSNR In	PSNR Out	SSIM Out
Clean	—	27.81	0.8713
Salt-and-pepper	16.38	27.48	0.8579
Gaussian blur	27.89	26.63	0.8131
Occlusion	13.56	21.69	0.7480
Overall	—	25.52	0.8128

H. Final Test-Set Evaluation

The evaluation contained 36,690 entries corresponding to 3,669 test images and ten conditions per image.

The model produced substantial restoration gains for salt-and-pepper corruption and occlusion. Gaussian blur behaved differently: low-severity blurred inputs already had PSNR of 32.44 dB, substantially above the autoencoder’s reconstruction quality. This behaviour is consistent with the information bottleneck constraint.

I. Qualitative Results

Quantitative metrics were complemented by visual inspection of the final reconstruction outputs.

The spatial bottleneck is capable of reconstructing recognizable image structure while removing substantial corruption. The improvement is particularly visible for salt-and-pepper noise.

J. Error Analysis and Failure Cases

Failure cases were examined separately because aggregate PSNR and SSIM do not fully describe where reconstruction errors occur.

The primary weakness was high-severity occlusion, where output PSNR decreased to 19.26 dB compared with 24.04 dB for low severity. The model also showed a smaller limitation on Gaussian blur. These results illustrate an important trade-off: a single compressed representation must simultaneously handle noise removal, deblurring, and missing-region reconstruction.

K. ONNX Export and Deployment

The final Task 1 model was exported to ONNX format. Five independent verification inputs were compared between PyTorch and ONNX models. The largest observed absolute difference was 3.874×10^{-7} , with mean absolute difference of approximately 4.99×10^{-8} .

TABLE VI
TASK 1 ONNX EXPORT AND VERIFICATION

Artifact	Result
Final checkpoint	Epoch 51
Checkpoint	final_v2/best.pt
ONNX model	task1_final_v2.onnx
Input	[batch, 3, 128, 128]
Output	[batch, 3, 128, 128]
ONNX checker	Passed
ONNX Runtime	Passed
Max absolute difference	3.874×10^{-7}
Mean absolute difference	4.99×10^{-8}
PyTorch/ONNX verification	Passed

L. Discussion

Preserving spatial structure in the latent representation was more important than simply increasing parameter count. The initial vector design achieved validation PSNR of 17.77 dB and SSIM of 0.440, whereas the final 16×16 spatial design achieved 23.69 dB and 0.749. On the deterministic test set,

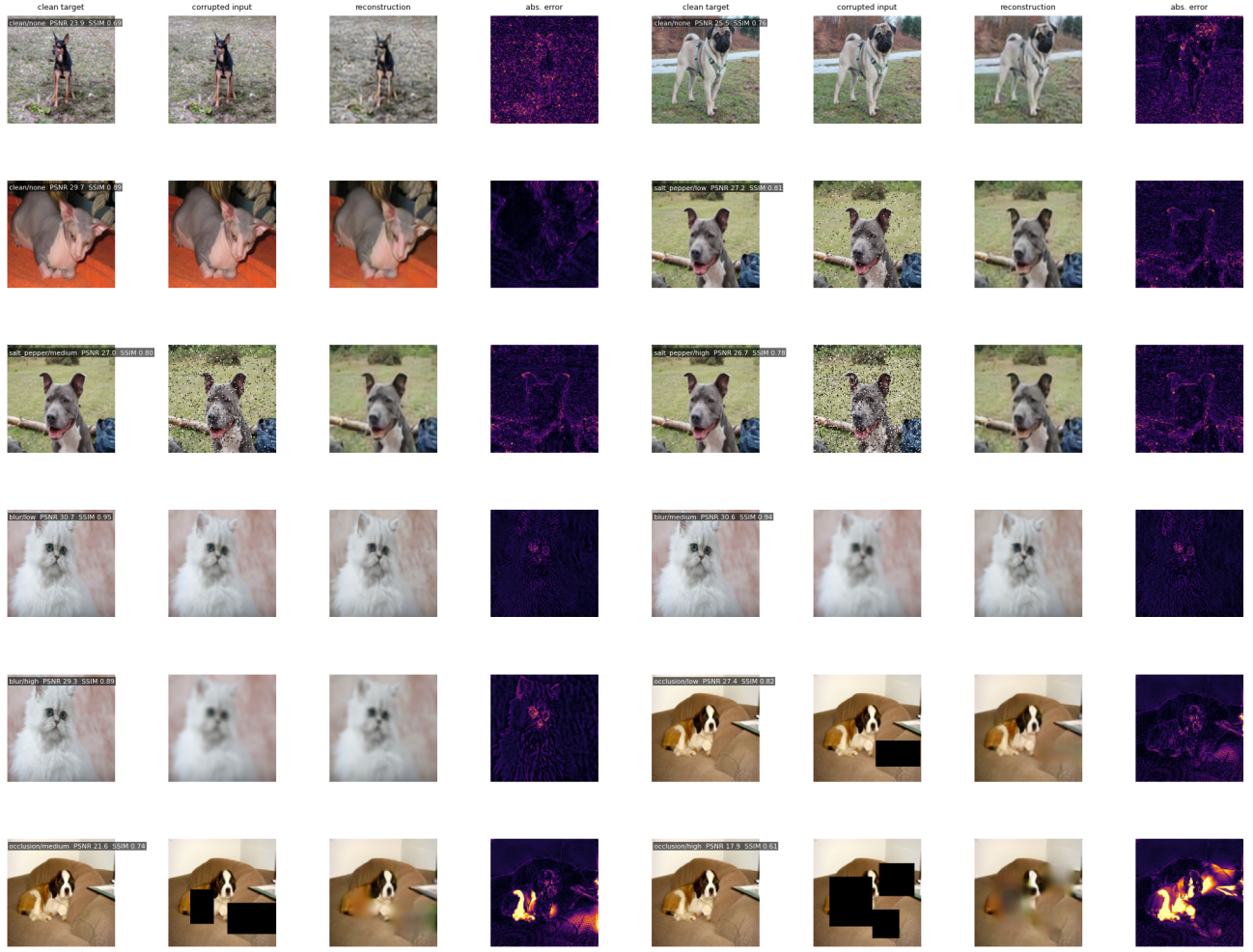


Fig. 7. Representative Task 1 failure cases and reconstruction errors.

the final model achieved overall PSNR of 25.52 dB and SSIM of 0.8128.

III. TASK 2: CORRUPTION CLASSIFICATION AND SPECIALIST IMAGE RECONSTRUCTION

A. Problem Formulation

The objective of Task 2 was to develop a corruption-aware image reconstruction pipeline capable of identifying corruption type and routing to a corresponding specialist reconstruction network. The system was designed as a two-stage pipeline: a convolutional classifier predicts the corruption class, and a routing component selects one of three specialist reconstruction networks.

B. Dataset Preparation

The processed dataset contained 3,680 images for training/validation and 3,669 clean test images. Training/validation split used seed 42, yielding 2,944 training and 736 validation images. Each test image was evaluated under ten conditions, resulting in 36,690 test records.

TABLE VII
DATASET CONFIGURATION FOR TASK 2

Property	Value
Training images	2,944
Validation images	736
Test images	3,669
Test records	36,690
Image resolution	128 × 128
Image channels	RGB
Corruption classes	4
Classes	Clean, Salt-Pepper, Blur, Occlusion
Split seed	42

C. Architecture Design

The system contains a corruption classifier followed by routing and three specialist reconstruction networks.

The classifier uses convolutional feature extraction with global average pooling and a fully connected classification layer using channels (16, 32, 64, 128). The specialist networks use U-Net-style encoder-decoder architecture with skip connections and residual correction: $\hat{x} = x + \Delta(x)$.

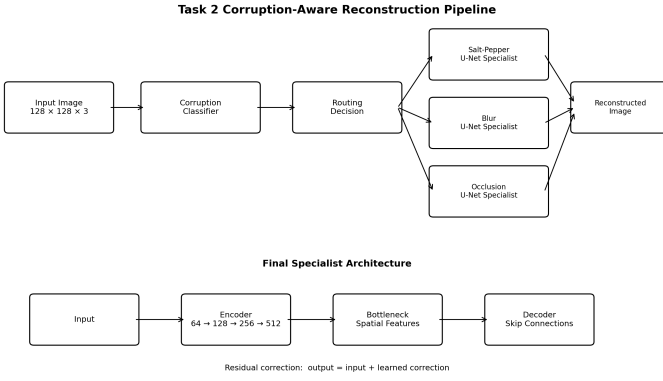


Fig. 8. Task 2 architecture showing corruption classification, routing, and specialist image reconstruction.

D. Corruption Classifier

The classifier was trained as a four-class model with labels: Clean, Salt-pepper, Blur, Occlusion. Optuna selected batch size 64, small channel configuration, dropout 0.2135, learning rate 2.58×10^{-4} , and weight decay 4.23×10^{-5} .

TABLE VIII
FINAL CORRUPTION CLASSIFIER CONFIGURATION

Parameter	Value
Architecture	Convolutional classifier
Channels	(16, 32, 64, 128)
Dropout	0.2135
Learning rate	0.00025779
Weight decay	0.00004228
Batch size	64
Epochs	30

The classifier reached 0.9728 accuracy and 0.9728 macro-F1 on validation data.

E. Specialist Reconstruction Networks

Three specialist networks were trained separately. All use U-Net architecture with encoder channels (64, 128, 256, 512) and spatial skip connections.

TABLE IX
FINAL SPECIALIST VALIDATION LOSSES

Specialist	Training Loss	Validation Loss
Salt-pepper	0.0075	0.0071
Blur	0.0187	0.0189
Occlusion	0.0354	0.0376

F. Routing Mechanism

During inference, the classifier predicts one of four corruption labels. Clean images pass through without reconstruction, while corrupted images route to the corresponding specialist:

$$k = \text{Classifier}(x), \quad \hat{x} = \text{Specialist}_k(x). \quad (2)$$

Both routed and oracle results were recorded for evaluation.

G. Test-Set Evaluation

The final system was evaluated on all 36,690 test records using L_1 , SSIM, and PSNR metrics.

TABLE X
FINAL TASK 2 RECONSTRUCTION RESULTS

Metric	Routed Specialist	Oracle Specialist
L_1	0.016354	0.015610
SSIM	0.908923	0.911154
PSNR	36.0503 dB	36.6027 dB

H. Per-Corruption Results

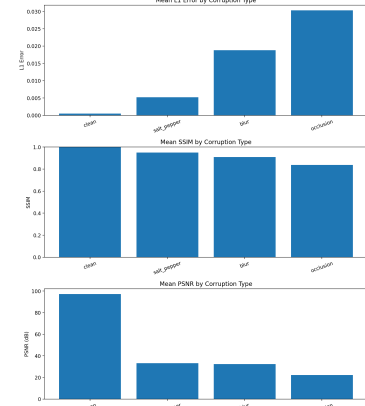


Fig. 9. Reconstruction performance broken down by corruption type.

I. Routing Results

The routing confusion matrix shows strong classification performance. Salt-and-pepper achieved 11,007 correct predictions with zero misclassifications. Blur achieved 10,413 correct predictions with 581 misclassified as clean. Occlusion achieved 10,345 correct predictions with 640 misclassified as clean.

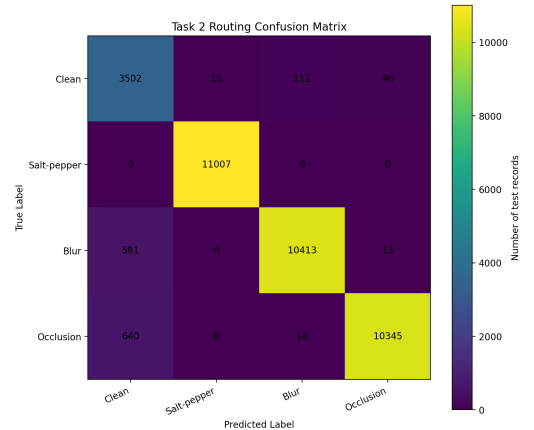


Fig. 10. Task 2 routing results showing corruption-class predictions.

J. Qualitative Results

The qualitative inspection showed a clear improvement over the original vector-bottleneck specialist architecture. The U-Net reconstruction retained substantially more spatial structure and was able to correct visible corruption while preserving the overall image content.

K. Error Analysis and Limitations

The occlusion specialist produced the highest validation loss among the three specialists. Unlike salt-pepper noise and blur, occlusion removes portions of the original image, meaning that missing content must be inferred from surrounding information. Another limitation is the use of separate specialists, which requires multiple models to be stored and deployed.

L. ONNX Export and Deployment

Four ONNX models were exported: classifier.onnx, salt.pepper.onnx, blur.onnx, and occlusion.onnx, each with corresponding .onnx.data files.

TABLE XI
TASK 2 FINAL ONNX DEPLOYMENT ARTIFACTS

Model	Deployment Files
Classifier	classifier.onnx, classifier.onnx.data
Salt-pepper	salt.pepper.onnx, salt.pepper.onnx.data
Blur	blur.onnx, blur.onnx.data
Occlusion	occlusion.onnx, occlusion.onnx.data

Model	ONNX validation	External data
classifier	Not validated	Present
salt_pepper	Not validated	Present
blur	Not validated	Present
occlusion	Not validated	Present

Fig. 11. ONNX export and validation results for the Task 2 deployment models.

M. Discussion

Task 2 established a complete corruption-aware reconstruction pipeline. The main architectural change was replacing vector-bottleneck design with U-Net architecture containing spatial skip connections and residual correction. Across all 36,690 test records, the routed system achieved $L_1 = 0.016354$, SSIM = 0.908923, and PSNR = 36.0503 dB.

IV. TASK 3: JOINTLY TRAINED SOFT MIXTURE-OF-EXPERTS

A. Problem Formulation

Task 3 extends Task 2 into a jointly trained soft mixture-of-experts (MoE) architecture. Instead of hard routing, the model produces a probability distribution over four reconstruction branches and combines outputs using learned weights. The four branches correspond to clean identity path, salt-and-pepper specialist, blur specialist, and occlusion specialist.

B. Dataset Preparation

The dataset configuration matches Task 2: 2,944 training images, 736 validation images, and 36,690 test records. For training the soft MoE, balanced corruption labels were generated with 920 examples per corruption class.

C. Architecture Design

The Task 3 architecture reuses Task 2 components while replacing hard routing with soft routing. The corruption classifier serves as the gate, while three U-Net specialists provide reconstruction branches. A fourth identity branch passes input directly.

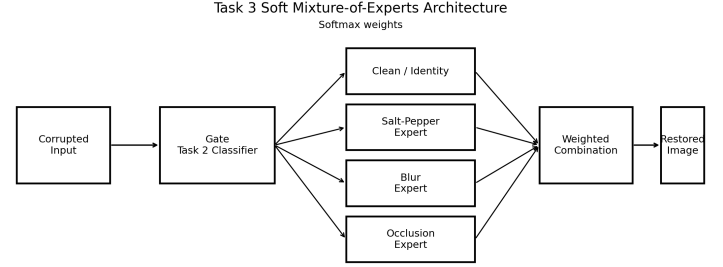


Fig. 12. Task 3 jointly trained soft mixture-of-experts architecture.

For input x , the gate produces logits $z = G(x)$. Routing weights use temperature-scaled softmax:

$$w_k = \frac{\exp(z_k/T)}{\sum_{j=1}^4 \exp(z_j/T)}. \quad (3)$$

Branch outputs are $y_0 = x$, $y_1 = E_{\text{salt}}(x)$, $y_2 = E_{\text{blur}}(x)$, $y_3 = E_{\text{occ}}(x)$. Final reconstruction:

$$\hat{x} = \sum_{k=0}^3 w_k y_k. \quad (4)$$

D. Gate and Expert Initialization

The gate was initialized from the Task 2 classifier checkpoint. Three experts were initialized from Task 2 specialist checkpoints, all using U-Net architecture with channels (64, 128, 256, 512) and residual correction.

E. Loss Function

The training objective combines reconstruction, classification, and routing balance:

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda_c \mathcal{L}_{\text{class}} + \lambda_b \mathcal{L}_{\text{balance}}, \quad (5)$$

where $\mathcal{L}_{\text{rec}} = \alpha \mathcal{L}_{L1} + (1 - \alpha) \mathcal{L}_{\text{SSIM}}$, $\mathcal{L}_{\text{class}} = \text{CE}(z/T, y)$, and $\mathcal{L}_{\text{balance}} = \sum_{k=1}^4 (\bar{w}_k - 0.25)^2$. Final selected values: $\alpha = 0.76779$, $\lambda_c = 0.014736$, $\lambda_b = 0.011993$, $T = 1.51527$.

F. Two-Stage Training

Stage 1: 3 warm-up epochs with gate trained and specialists frozen. Stage 2: joint fine-tuning of all components with learning rate 1.52376×10^{-5} and batch size 8. BatchNorm layers kept in evaluation mode.

G. Hyperparameter Optimization

Optuna searched learning rate (10^{-5} to 3×10^{-4}), temperature (0.5 to 3.0), λ_c (0.01 to 0.5), λ_b (10^{-3} to 0.1), and α (0.5 to 0.95).

TABLE XII
SELECTED TASK 3 HYPERPARAMETERS

Parameter	Selected Value
Learning rate	0.0000152376
Temperature	1.51527
Classification weight λ_c	0.014736
Balance weight λ_b	0.011993
Reconstruction coefficient α	0.767792
Batch size	8

H. Training Results

Best checkpoint occurred after warm-up and 18 joint-training epochs (total epoch 21). Best validation score was 0.07154.

TABLE XIII
SELECTED TASK 3 FINAL-TRAINING CHECKPOINTS

Stage	Epoch	Val. L_1	Val. Score
Warm-up	3	0.0124	0.0829
Joint	1	0.0119	0.0774
Joint	5	0.0111	0.0745
Joint	10	0.0108	0.0725
Joint	14	0.0108	0.0724
Joint	18	0.0108	0.0715

I. Routing Analysis

Routing behaviour was evaluated on all 736 validation images.

TABLE XIV
TASK 3 MEAN ROUTING WEIGHTS ON VALIDATION SET

True Class	Clean	Salt-Pep.	Blur	Occl.
Clean	0.898	0.006	0.066	0.030
Salt-Pepper	0.005	0.992	0.001	0.002
Blur	0.035	0.001	0.958	0.006
Occlusion	0.033	0.002	0.007	0.958

The routing matrix shows strong diagonal behaviour. Across all validation samples, 90.49% had dominant routing weight greater than 0.9. Mean off-diagonal routing mass was 0.0487, and mean routing entropy was 0.1586. The routing distribution was stable and non-collapsed.

J. Expert Activity

Average routing weights were (0.2529, 0.2262, 0.2962, 0.2247) for clean, salt-pepper, blur, and occlusion branches. Values remain balanced around 0.25.

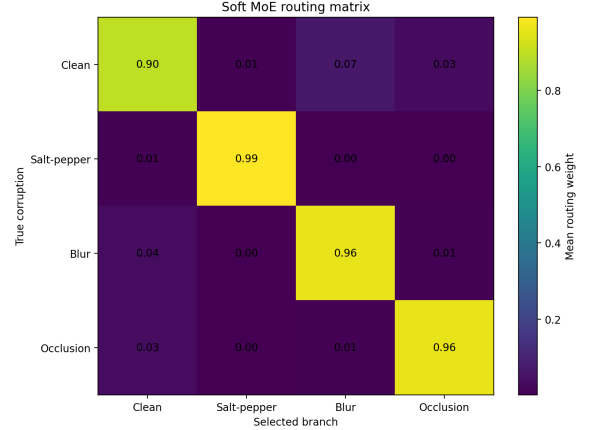


Fig. 13. Task 3 routing heatmap showing the mean soft routing weights for each true corruption class.

TABLE XV
FINAL TASK 3 RESULTS OVER ALL 36,690 TEST RECORDS

Metric	No Restoration	Soft MoE
L_1	0.050542	0.014086
SSIM	0.745837	0.918089
PSNR	—	36.4719 dB

TABLE XVI
TASK 3 RESULTS BY CORRUPTION TYPE

Corruption	Method	L_1	SSIM	PSNR
Clean	No rest.	0.000000	1.000000	∞
Clean	Soft MoE	0.000100	0.999636	90.3735
Salt-Pepper	No rest.	0.043340	0.598443	16.3838
Salt-Pepper	Soft MoE	0.003680	0.961358	35.0805
Blur	No rest.	0.028541	0.784247	27.8911
Blur	Soft MoE	0.017709	0.881827	33.0844
Occlusion	No rest.	0.096592	0.770099	13.5618
Occlusion	Soft MoE	0.025531	0.883899	23.2837

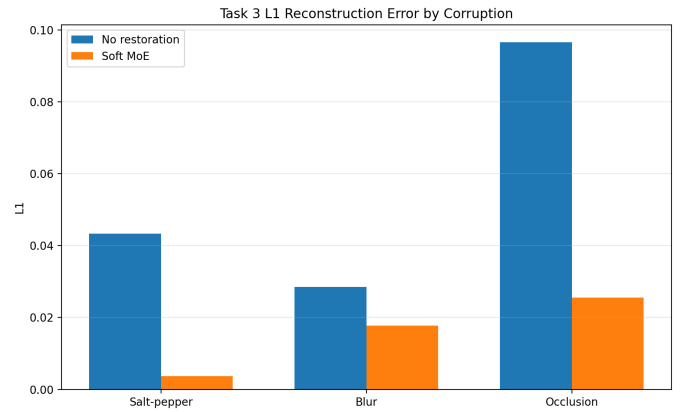


Fig. 14. Task 3 L_1 reconstruction error by corruption type.

TABLE XVII
COMPARISON BETWEEN TASK 2 AND TASK 3

Metric	Task 2 Routed	Task 3 Soft MoE
L_1	0.016354	0.014086
SSIM	0.908923	0.918089
PSNR	36.0503 dB	36.4719 dB

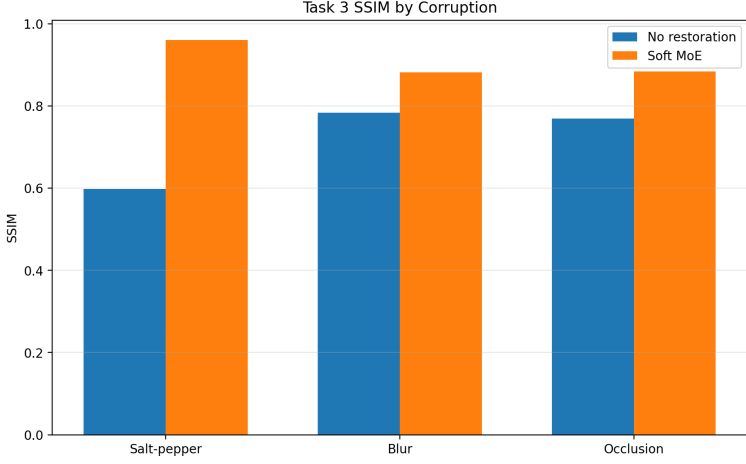


Fig. 16. Qualitative Task 3 reconstruction examples showing the corrupted input, Soft MoE reconstruction, and clean target.

K. Test-Set Evaluation

L. Per-Corruption Results

M. Qualitative Results

N. Comparison with Task 2

The jointly trained soft MoE improves both average pixel reconstruction error and structural similarity compared with the Task 2 routed system.

O. ONNX Export and Deployment

The final Soft MoE model was exported as a single ONNX graph accepting input $(B, 3, 128, 128)$ and producing three outputs: restored image, soft routing weights, and gate logits. The model uses ONNX opset 17 and supports dynamic batch dimension.

TABLE XVIII
TASK 3 ONNX DEPLOYMENT ARTIFACT

Artifact	Description
task3_soft_moe.onnx	Complete Soft MoE inference graph
Input	RGB image, $3 \times 128 \times 128$
Outputs	Restored image, routing weights, logits
Batch dimension	Dynamic
Opset	17

P. Discussion

Task 3 extended the corruption-aware pipeline into a jointly trained soft mixture-of-experts. Routing analysis showed strong specialization with average diagonal weights of 0.898 (clean), 0.992 (salt-pepper), 0.958 (blur), and 0.958 (occlusion). Across all 36,690 test records, the Soft MoE achieved $L_1 = 0.014086$, SSIM = 0.918089, and PSNR = 36.4719 dB.

V. TASK 4: CONDITIONAL GAN FOR PHOTO-TO-SKETCH TRANSLATION

A. Problem Formulation

The objective of Task 4 was to learn a conditional image-to-image translation model that converts facial photographs to sketches while preserving identity and structural information. The task was formulated as a conditional Generative Adversarial Network (cGAN), where the generator receives both a photograph and a sketch-style label. A conditional PatchGAN discriminator distinguishes generated sketches from real sketches.

B. Dataset Preparation

The FS2K dataset provides paired photograph-sketch images with style information. The training split contained 1,058 paired samples, and the test split contained 1,046 samples. Images were resized to 256×256 pixels and normalized to $[-1, 1]$.

TABLE XIX
FS2K DATASET CONFIGURATION FOR TASK 4

Property	Value
Training samples	1,058
Test samples	1,046
Image resolution	256×256
Input	Photograph
Target	Sketch
Condition	Sketch style label

C. Architecture Design

The model consists of a conditional generator and conditional PatchGAN discriminator.

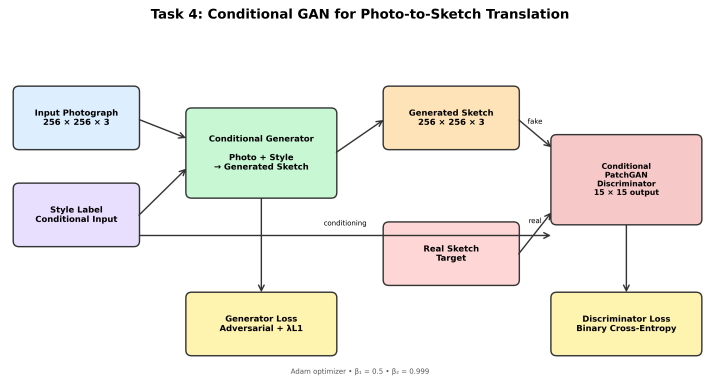


Fig. 17. Task 4 conditional GAN architecture.

With batch size 2, the generator produces tensors of shape $[2, 3, 256, 256]$, and the discriminator produces $[2, 1, 15, 15]$, confirming PatchGAN operation over local regions.

D. Loss Functions

The generator objective:

$$\mathcal{L}_G = \mathcal{L}_{adv} + \lambda_{L1} \mathcal{L}_{L1}, \quad (6)$$

where $\mathcal{L}_{adv}$ encourages realistic sketches and $\mathcal{L}_{L1}$ measures pixel-level difference. The discriminator uses binary cross-entropy for real and generated samples.

E. Training Procedure

The model used Adam optimizer with $\beta_1 = 0.5$ and $\beta_2 = 0.999$. Mixed-precision training was enabled when CUDA was available. Final training used 50 epochs.

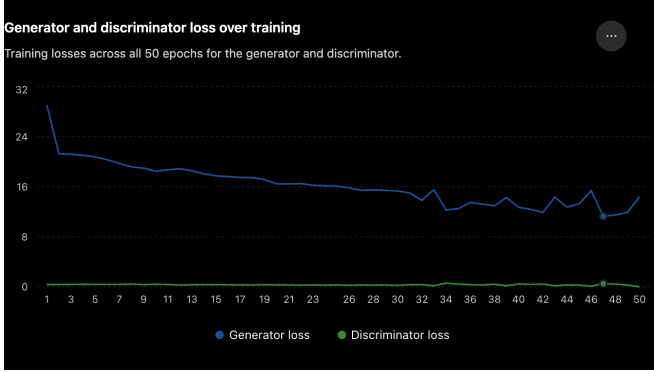


Fig. 18. Training curves for the Task 4 conditional GAN.

F. Optuna Hyperparameter Search

Optuna searched generator learning rate (5×10^{-5} to 5×10^{-4}), discriminator learning rate (5×10^{-5} to 5×10^{-4}), λ_{L1} (50 to 150), and batch size $\{4, 8\}$. Trial 4 produced lowest generator loss of 21.2045 with generator LR $\approx 2.05 \times 10^{-4}$, discriminator LR $\approx 6.89 \times 10^{-5}$, $\lambda_{L1} = 79.214465$, and batch size 8.

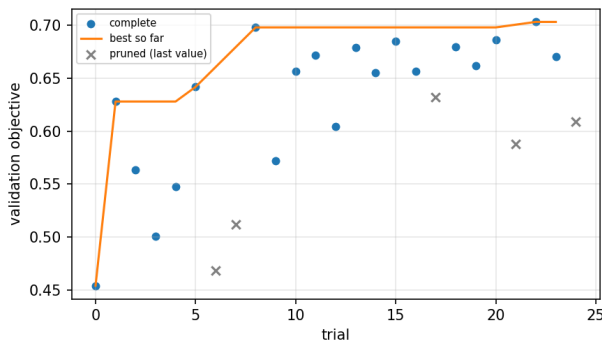


Fig. 19. Optuna optimization history for the Task 4 search.

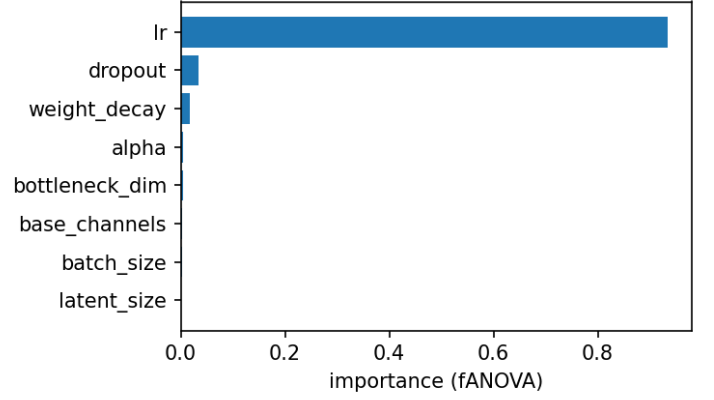


Fig. 20. Optuna hyperparameter importance for the Task 4 search.

TABLE XX
FINAL TEST-SET RESULTS FOR TASK 4

Metric	Value
L_1	0.110794
MSE	0.041651
PSNR	15.318455
SSIM	0.481005

G. Final Test-Set Evaluation

H. Qualitative Results

I. Error Analysis and Failure Cases

A likely limitation is the tension between pixel-level reconstruction and adversarial realism. The L_1 component encourages direct reproduction of the reference image, while the adversarial component encourages visually plausible local structure. The relatively small paired training set and inherent difficulty of reproducing sketch-specific line structure from photographs are primary limitations.

J. ONNX Export and Deployment

TABLE XXI
TASK 4 FINAL ARTIFACTS

Artifact	Result
Final checkpoint	Epoch 50
Checkpoint size	323.93 MB
ONNX generator	Successfully exported
ONNX input	photo, style
ONNX output	sketch
ONNX size	0.01 MB
ONNX validation	Passed

K. Discussion

Task 4 established a complete conditional GAN pipeline for photo-to-sketch translation. The final quantitative results indicate moderate structural correspondence, with SSIM of 0.481005 and L_1 error of 0.110794. The main limitation is the relatively small paired training set and inherent difficulty of reproducing sketch-specific line structure.

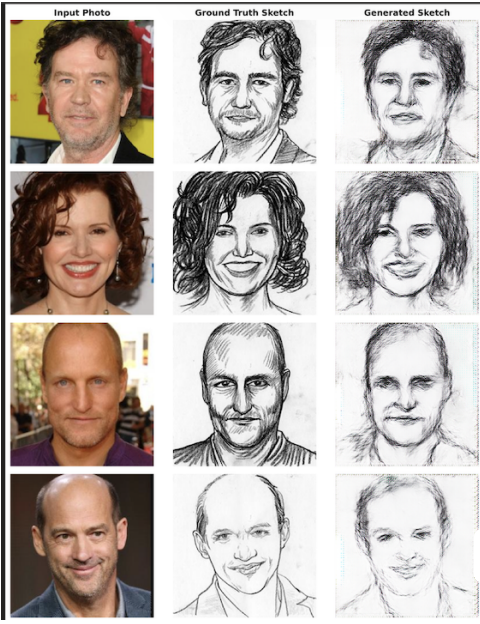
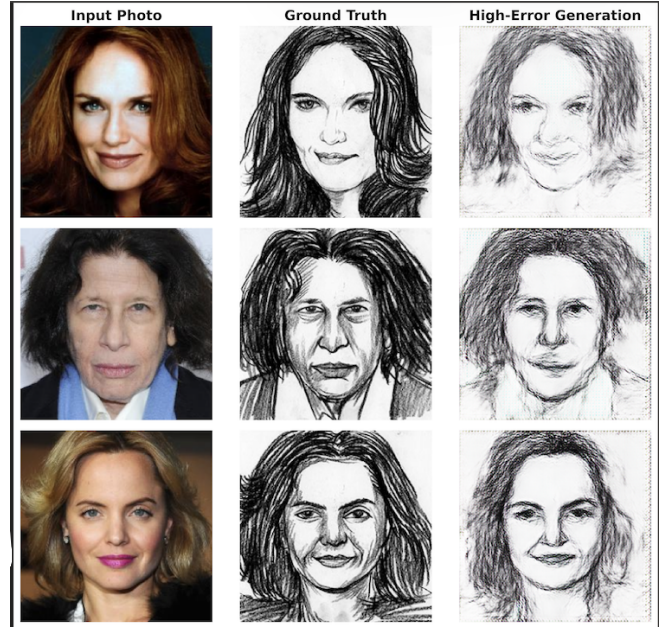


Fig. 22. Representative failure cases produced by the Task 4 generator.



TASK 4 – ONNX DEPLOYMENT VALIDATION

- ✓ ONNX export successful
- ✓ Input 1: photo
- ✓ Input 2: style
- ✓ Output: sketch
- ✓ ONNX model check passed

Final generator:
Epoch: 50
Checkpoint size: 323.93 MB
ONNX model size: 0.01 MB

Fig. 24. ONNX export and numerical validation results for the Task 4 generator.

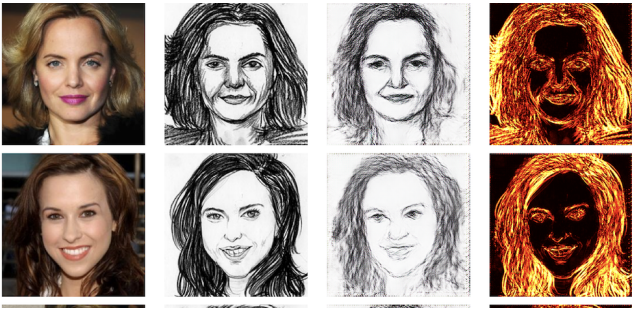


Fig. 23. Absolute reconstruction error maps for representative Task 4 translations.

VI. CONCLUSION

This report presented four interconnected tasks addressing image restoration and translation. Task 1 established that spatial latent representations substantially outperform vector bottlenecks for universal image restoration, achieving PSNR of 25.52 dB and SSIM of 0.8128. Task 2 introduced corruption-aware routing with specialist networks, achieving routed PSNR of 36.05 dB. Task 3 extended this to jointly trained soft mixture-of-experts, improving to PSNR of 36.47 dB and SSIM

of 0.918. Task 4 addressed photo-to-sketch translation using conditional GANs, achieving SSIM of 0.481.

Key findings include: (1) preserving spatial structure in latent representations is more important than increasing parameter count; (2) U-Net skip connections substantially improve reconstruction quality; (3) joint optimization of routing and reconstruction networks improves over hard routing; and (4) soft mixture-of-experts provides stable, non-collapsed routing while maintaining strong specialization. All models were successfully exported to ONNX format for deployment.

REFERENCES

- [1] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
- [2] I. Goodfellow et al., “Generative Adversarial Nets,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 2672–2680.
- [3] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, “Image-to-Image Translation with Conditional Adversarial Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1125–1134.
- [4] T. Akiba et al., “Optuna: A Next-generation Hyperparameter Optimization Framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.

APPENDIX A STITCH DESIGN SCREENSHOTS

This appendix presents the frontend design screenshots created in Stitch and later implemented in React.

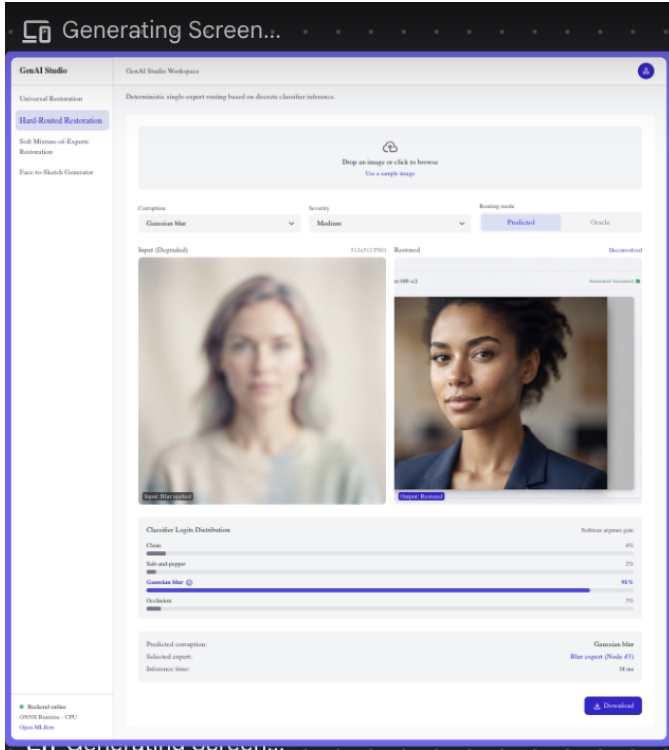


Fig. 1. Stitch frontend design — Screen 1.

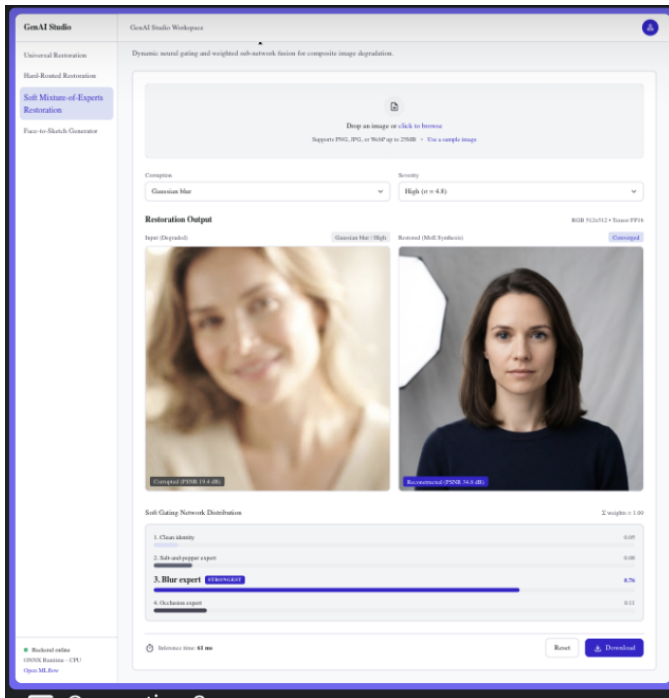


Fig. 2. Stitch frontend design — Screen 2.

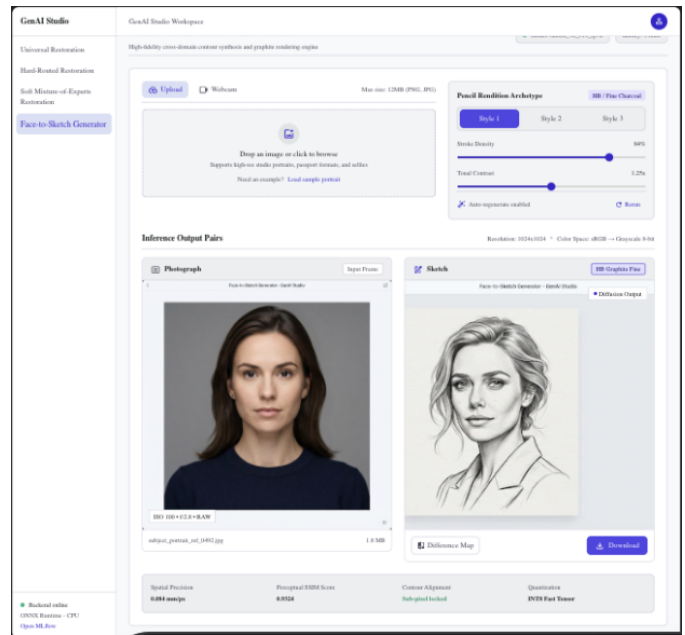


Fig. 3. Stitch frontend design — Screen 3.

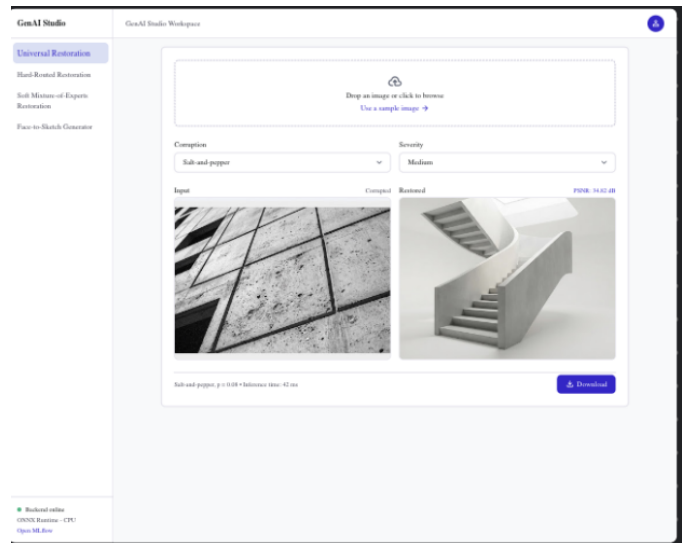


Fig. 4. Stitch frontend design — Screen 4.

APPENDIX B DEMONSTRATION VIDEO

A complete walkthrough of the assignment is available at the following link:

TABLE I
DEMONSTRATION VIDEO.

Field	Value
Title	Generative AI Assignment 01 — Task Demo
Platform	YouTube
Link	https://youtu.be/3CN5Qucrnua

APPENDIX C

AI APPENDICES

This appendix documents the use of AI-assisted tools during the development of this assignment.

TABLE II
AI TOOLS USED DURING DEVELOPMENT.

Tool	Purpose	Used For
ChatGPT	Debugging, code review	All tasks
Claude	Report writing, LaTeX formatting	All tasks

A. Usage Summary

leftmargin=1.4em,itemsep=0.25em

- **Code assistance:** AI tools were used to debug training loops, diagnose errors, and suggest architectural improvements. All suggested code was reviewed and tested before inclusion.
- **Report drafting:** AI tools assisted with structuring the report, refining technical language, and formatting tables and figures in LaTeX.
- **Experiment analysis:** AI tools helped interpret evaluation metrics and identify patterns in the reconstruction results.
- **Verification:** All AI-generated content was independently verified through testing, re-running experiments, and comparing against the original objectives.

B. Original Work Statement

All model architectures, training procedures, hyperparameter searches, frontend design, and experimental evaluations were designed and executed by the author. AI tools served in a supporting role for debugging, writing assistance, and code review. The final implementation, results, and conclusions are the author's own work.

APPENDIX A CODE REPOSITORY

The complete source code for all four tasks, including training scripts, evaluation utilities, ONNX export pipelines, and the React frontend, is available at:

TABLE I
GITHUB REPOSITORY.

Field	Value
Repository	genai-assignment1
Owner	Ayesha-Noor-04
Link	https://github.com/Ayesha-Noor-04/genai-assignment1

A. Quick Start with Docker

The fastest way to run the entire project is through Docker. The repository includes Dockerfiles for each task and a `docker-compose.yml` that orchestrates the full pipeline.

1) *Prerequisites:* leftmargin=1.4em,itemsep=0.25em

- Docker Engine 20.10 or newer
- Docker Compose v2 (included with modern Docker Desktop)
- NVIDIA Container Toolkit (for GPU acceleration)
- Approximately 15 GB of free disk space

2) *Clone and Launch:*

```
git clone https://github.com/Ayesha-Noor-04/genai-assignment1.git
cd genai-assignment1
docker compose up --build
```

This single command builds all images, installs dependencies, and starts the full stack. Training outputs, checkpoints, and ONNX artifacts are written to the mounted `./outputs` directory on the host.

3) *Running Individual Tasks:* To run a single task without spinning up the whole stack:

```
# Task 1
docker compose run --rm task1

# Task 2
docker compose run --rm task2

# Task 3
docker compose run --rm task3

# Task 4 (training + evaluation + ONNX export)
docker compose run --rm task4
```

4) *GPU Support:* If the NVIDIA Container Toolkit is installed, the containers will automatically use the GPU. To verify:

```
docker compose run --rm task4 nvidia-smi
```

The output should list the available GPU. If no GPU is detected, the containers fall back to CPU execution, which is substantially slower.

5) *Stopping and Cleanup:*

```
# Stop all running containers
docker compose down

# Remove images and volumes as well
docker compose down --rmi all --volumes
```

B. Frontend

The React frontend corresponding to the Stitch designs runs as a separate service:

```
docker compose up frontend
```

It is exposed on `http://localhost:3000` by default.

C. Manual Installation (Fallback)

If Docker is unavailable, each task directory contains a standalone `requirements.txt` and can be executed directly:

```
cd taskN
pip install -r requirements.txt
python -m taskN.src.train --data-root /path/to/data
```